

SQL-Übersicht (MySQL/MariaDB)

DDL, DML, DQL, Constraints, Indizes, Views, Prozeduren, Trigger, Rechte, Performance

1. Datentypen (Auswahl)

- Ganzzahl: TINYINT, SMALLINT, INT, BIGINT
- Fließkomma: FLOAT, DOUBLE
- Exakte Zahlen: DECIMAL(p, s)
- Zeichen: CHAR(n), VARCHAR(n), TEXT
- Binär: BLOB, VARBINARY(n)
- Datum/Zeit: DATE, TIME, DATETIME, TIMESTAMP
- Boolean: BOOLEAN (= TINYINT(1))
- JSON: JSON

2. DDL – Tabellen und Schema

2.1 CREATE TABLE

```
CREATE TABLE kunden (  
  kundennr INT PRIMARY KEY AUTO_INCREMENT,  
  nachname VARCHAR(30) NOT NULL,  
  gebdatum DATE,  
  email VARCHAR(100) UNIQUE,  
  plz CHAR(5),  
  ort VARCHAR(50) DEFAULT 'n.a.'  
);
```

- Tabellenkopie: CREATE TABLE neu LIKE alt;
- Daten kopieren: CREATE TABLE neu AS SELECT ...;

2.2 Constraints

- Primärschlüssel: PRIMARY KEY (sp1)
- Eindeutig: UNIQUE (sp1)
- Fremdschlüssel: FOREIGN KEY (fk) REFERENCES t(pk)
- Standardwert: DEFAULT wert
- Pflichtfeld: NOT NULL
- Prüfung: CHECK (bedingung)

2.3 ALTER TABLE

Tabellenname ändern

```
ALTER TABLE alt RENAME TO neu;
```

Spalte hinzufügen

```
ALTER TABLE kunden  
ADD gebdatum DATE;
```

Spalte löschen

```
ALTER TABLE kunden  
DROP gebdatum;
```

Datentyp ändern

```
ALTER TABLE kunden  
MODIFY plz CHAR(5);
```

Primärschlüssel hinzufügen

```
ALTER TABLE t  
ADD PRIMARY KEY (id);
```

Fremdschlüssel hinzufügen

```
ALTER TABLE bestellungen  
ADD CONSTRAINT fk_kunde  
FOREIGN KEY (kundennr)  
REFERENCES kunden(kundennr);
```

2.4 DROP / TRUNCATE

- Tabelle löschen: DROP TABLE t;
- Tabelle leeren: TRUNCATE TABLE t;
- Tabelle umbenennen: RENAME TABLE alt TO neu;

3. Indizes

```
CREATE INDEX idx_name  
ON t(spalte);  
  
CREATE UNIQUE INDEX idx_u  
ON t(spalte);  
  
DROP INDEX idx_name ON t;
```

4. DML – Daten bearbeiten

4.1 INSERT

```
INSERT INTO t (sp1, sp2)  
VALUES (1, 'A'), (2, 'B');
```

4.2 UPDATE

```
UPDATE t  
SET sp1 = 'Neu'  
WHERE id = 1;
```

4.3 DELETE

```
DELETE FROM t  
WHERE id = 1;
```

4.4 Transaktionen

```
START TRANSACTION;  
  
UPDATE konto  
SET saldo = saldo - 100  
WHERE id = 1;  
  
COMMIT;  
ROLLBACK;
```

5. SELECT – Abfragen

5.1 Grundform

```
SELECT sp1, sp2
FROM t
WHERE bedingung
GROUP BY sp1
HAVING COUNT(*) > 1
ORDER BY sp1 DESC
LIMIT 10;
```

5.2 JOINS

```
SELECT k.nachname,
b.bestellnr
FROM kunden k
INNER JOIN bestellungen b
ON b.kundennr = k.kundennr;
```

- INNER JOIN: nur passende Datensätze
- LEFT JOIN: alle links, passende rechts
- RIGHT JOIN: alle rechts, passende links
- CROSS JOIN: kartesisches Produkt

5.3 Aggregation

```
SELECT kundennr,
COUNT(*) AS anzahl,
SUM(betrag) AS summe
FROM bestellungen
GROUP BY kundennr;
```

5.4 Subqueries

```
SELECT *
FROM kunden
WHERE id IN (
SELECT kundennr
FROM bestellungen
);
```

6. Views

```
CREATE VIEW v_kunden AS
SELECT *
FROM kunden;

DROP VIEW v_kunden;
```

7. Prozeduren / Trigger

7.1 Procedure

```
DELIMITER //

CREATE PROCEDURE p()
BEGIN
SELECT * FROM kunden;
END //

DELIMITER ;
```

7.2 Trigger

```
CREATE TRIGGER trg_bi
BEFORE INSERT ON t
FOR EACH ROW
SET NEW.erstellt_am = NOW();
```

8. Sicherheit

```
CREATE USER 'u'@'localhost'
IDENTIFIED BY 'pw';

GRANT SELECT, INSERT
ON db.*
TO 'u'@'localhost';

REVOKE INSERT
ON db.*
FROM 'u'@'localhost';
```

9. Performance

- EXPLAIN SELECT ...
- SHOW INDEX FROM t;
- SHOW CREATE TABLE t;
- SHOW PROCESSLIST;

10. Best Practices

- Geldwerte: DECIMAL statt FLOAT
- VARCHAR: variable Länge
- CHAR: feste Länge
- IS NULL statt = NULL
- Für Foreign Keys: InnoDB verwenden
- Vor DROP und ALTER immer Backup erstellen